



Згорнути меню

Створення Docker-імеджу для ML-моделей

Контейнеризація моделі на базі PyTorch

У цьому прикладі ми розглянемо, як створити **Docker-імедж для нейронної мережі**, зібраної у фреймворку **PyTorch**. Ми оберемо типову модель для задачі класифікації зображень, запакуємо її в TorchScript, напишемо скрипт для inference, додамо всі залежності — і зрештою отримаємо повноцінний контейнер, який можна запускати в будь-якому середовищі.

Ми створимо невеликий проєкт, який:

- Використовує готову претреновану модель з PyTorch,
- Обробляє зображення й робить inference,
- Зберігає модель у форматі TorchScript (`.pt`),
- Виконується всередині Docker-контейнера.

Це класичний use case для реального ML-сервісу: у вас уже є модель, вам потрібно запускати її стабільно та незалежно від середовища.

Що таке PyTorch?

PyTorch — це один із найпопулярніших фреймворків для **глибокого навчання**, розроблений у **Facebook AI Research (FAIR)**. Він став стандартом у дослідницьких проєктах, академії та продакшені завдяки своїй:

- Зрозумілій Python-подібній логіці (imperative style programming),
- Гнучкості в побудові нейромереж,
- Підтримці CPU/GPU,
- Великій кількості готових моделей (через `torchvision`, `torchaudio` тощо),
- Активній спільноті та відкритому вихідному коду.

Імперативне програмування — парадигма програмування, згідно з якою описується процес отримання результатів як послідовність інструкцій зміни стану

програми.

А ліцензія?

PyTorch — **повністю безкоштовний** і має ліцензію BSD. Це означає, що можна:

- Використовувати його у власних і комерційних проєктах,
- Розгортати на сервері або в хмарі,
- Не турбуватись про юридичні обмеження.

Далі ми розглянемо, як **контейнеризувати ML-модель**, створену у фреймворку **PyTorch**, тобто як підготувати її до запуску в будь-якому середовищі через Docker.

Наша основна ціль — **створити повноцінний Docker-імедж**, який:

- Містить уже натреновану модель,
- Приймає зображення на вхід,
- Виконує передбачення (inference),
- Запускається командою типу `docker run` без жодних зовнішніх залежностей.

Це типовий use case у реальному житті:



Ви маєте готову модель і хочете запускати її як частину сервісу, job'и в Kubernetes або через API.

Для цього вам потрібен **ізолюваний, стабільний контейнер**, тобто Docker-імедж.

📌 Структура проєкту:

```
pytorch-image/
├── save_model.py           # створює model/traced_model.pt
├── app/
│   ├── inference.py       # скрипт для запуску
│   └── requirements.txt   # залежності Python
├── model/
│   └── traced_model.pt    # буде створений автоматично
├── Dockerfile             # опис контейнера
└── example.jpg            # тестове зображення
```

Ідемо покроково:

КРОК 1. Створити проєкт і каталоги

```
mkdir pytorch-image-classifier
cd pytorch-image-classifier
```

Пояснення:

- `mkdir` створює нову папку з назвою `pytorch-image-classifier`. Це наш робочий каталог.
- `cd` переходить у цю папку. Усі подальші файли створюються всередині неї.

КРОК 2. Створити .pt модель (TorchScript)

Файл: `save_model.py`

```
import torch
import torchvision.models as models
import os

# Створення папки для збереження моделі
os.makedirs("model", exist_ok=True)

# Завантаження попередньо натренованої моделі ResNet18
model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
model.eval() # Перехід у режим оцінки (inference)

# Створення "манекена" для трасування моделі
dummy_input = torch.rand(1, 3, 224, 224)

# Трасування моделі в TorchScript
traced_model = torch.jit.trace(model, dummy_input)

# Збереження моделі
traced_model.save("model/traced_model.pt")
print("✅ Model saved to model/traced_model.pt")
```

Пояснення:

- `torch.jit.trace` перетворює модель на TorchScript — формат, придатний для продакшену.
- `model.eval()` вимикає навчальні режими, як-от Dropout або BatchNorm.
- Створюємо `dummy`-вхід — випадкове зображення потрібної форми.
- TorchScript дозволяє запускати модель **без оригінального коду Python**, що зручно в Docker.

Запуск команди:

```
python3 save_model.py
```

КРОК 3. Написати інференс-скрипт

Файл: `app/inference.py`

```
import torch
from torchvision import transforms
from PIL import Image
import sys

model = torch.jit.load("model/traced_model.pt")
model.eval()

preprocess = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor()
])

def predict(image_path):
    image = Image.open(image_path).convert("RGB")
    input_tensor = preprocess(image).unsqueeze(0)
    with torch.no_grad():
        output = model(input_tensor)
        class_id = output.argmax(dim=1).item()
        print(f"🧠 Predicted class ID: {class_id}")

if __name__ == "__main__":
    predict(sys.argv[1])
```

Пояснення:

- Завантажуємо TorchScript-модель.
- Встановлюємо preprocessing для зображення: зміна розміру, кадрування, перетворення в тензор.
- Функція predict() приймає шлях до зображення, передає його в модель і виводить ID передбаченого класу.
- sys.argv[1] дозволяє передавати шлях до зображення через CLI.

КРОК 4. Зазначити залежності

Файл: app/requirements.txt

```
torch
torchvision
pillow
```

Пояснення:

- torch — для роботи з PyTorch і TorchScript.
- torchvision — для моделей і трансформацій.

- pillow — для обробки зображень.

КРОК 5. Написати Dockerfile

Файл: Dockerfile

```
FROM python:3.11-slim

RUN apt-get update && apt-get install -y --no-install-recommends \\\
    libgl1-mesa-glx libsm6 libxrender1 libxext6 \\\
    && apt-get clean && rm -rf /var/lib/apt/lists/*

WORKDIR /app

COPY app/ /app/
COPY model/ /app/model/
COPY app/requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

ENTRYPOINT ["python", "inference.py"]
```

Пояснення:

- Використовуємо легкий образ `python:3.11-slim`.
- Встановлюємо системні залежності, необхідні для Pillow і Torch.
- Вказуємо робочий каталог `/app`.
- Копіюємо код і модель усередину контейнера.
- Встановлюємо Python-бібліотеки.
- Встановлюємо команду за замовчуванням — запуск `inference.py` при старті контейнера.

КРОК 6. Завантажити тестове зображення

```
wget
<https://upload.wikimedia.org/wikipedia/commons/2/26/YellowLabradorLo
oking_new.jpg> -O example.jpg
```

Пояснення:

- Завантажуємо приклад зображення з Вікіпедії, яке будемо використовувати для тестування моделі.

КРОК 7. Зібрати Docker-образ

```
docker build -t pytorch-infer .
```

Пояснення:

- `docker build` створює Docker-образ.
- `t pytorch-infer` дає йому ім'я.
- `.` означає, що Dockerfile знаходиться в поточному каталозі.

КРОК 8. Запустити контейнер

До наступного блоку

